

Exercise 1-A (Summer 2026)

6.0 VU Advanced Database Systems

Information

Exercise Sheets

This exercise consists of **three exercise sheets** which will be available in TUWEL on:

- Exercise Sheet 1-A (PL/pgSQL): March 2nd
- Exercise Sheet 1-B (Datalog): March 4th
- Exercise Sheet 1-C (Common Table Expressions): March 6th

General

Work through **all three exercise sheets** and compile your answers in a **single report**. Submit the report and your source files as a **single ZIP (.zip)** file on TUWEL. The ZIP (.zip) file must contain:

- one PDF (.pdf) file,
- SQL (.sql) files containing your PostgreSQL commands (per exercise in sheet),
- Datalog (.dl) files containing your Datalog commands (per exercise in sheet).

SQL/Datalog file naming format: ex-⟨Exercise Sheet⟩-⟨Exercise Number⟩.⟨Question Number⟩.{sql/dl}

Submit the ZIP (.zip) file (max. 20 MB) and register for an exercise interview. **You can only receive points for this exercise if you attend the interview.** We expect you to explain your work in your report, i.e., it is not enough to only write the final answers. Show how you arrived at your answers and, where applicable, discuss your results. **We will not accept any handwritten reports!**

This exercise sheet involves working on a PostgreSQL DBMS and Datalog systems. For these exercises we provide you with web tools. For this particular exercise sheet you will use the PostgreSQL web tool, which is described in detail below.

Deadline

at latest **March 24th 9:00** Upload your submission on TUWEL
Do not forget! ⇒ Register for an exercise interview in TUWEL

Exercise Interviews

During the exercise interview, the correctness of your solution as well as your understanding of the underlying concepts will be assessed.

The scoring of your submission is primarily based on your performance at the exercise interview. Therefore it is possible (in extreme cases) to get 0 points even though the submitted solution was technically correct.

Please be punctual for your exercise interview. Otherwise we cannot guarantee that your full solution can be graded in your assigned time slot. Remember to have your student id with you for the exercise interview. It is not possible to score your solution without an id.

TUWEL Forum

You can use the course forum on TUWEL for clarifying questions regarding the exercise sheets. Please do not post your solutions (even partial) on the forum.

Postgres environment

For this exercise, we provide a web tool with a locally running PostgreSQL 16. You can access the tool at <https://dbai.tuwien.ac.at/adbs/>. On that page, click the *Open PostgreSQL Tool* button, to launch the PostgreSQL interface, as shown in Figure 1.

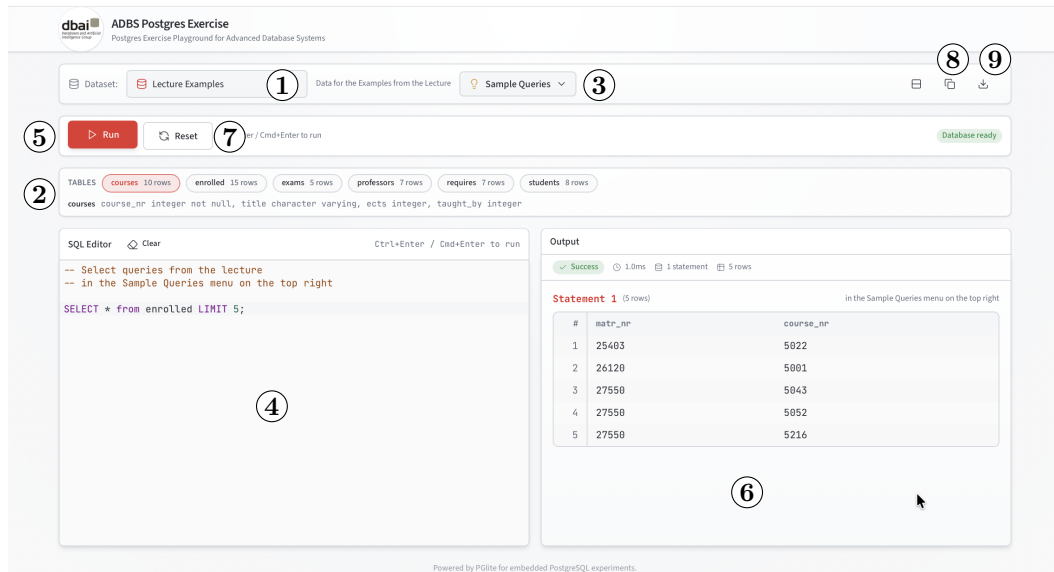


Figure 1: The PostgreSQL Tool

- ① You can use this drop-down menu to load one of the listed datasets.
- ② Once a dataset is loaded, the tables in the dataset and their details will appear in the information bar. Selecting a table will display its schema details.
- ③ In some cases, sample queries are provided for you to explore. You can choose an existing sample query from this drop-down menu and it will appear in ④.
- ④ You can type new queries or modify the sample queries using this editor.
- ⑤ Click this Run button to execute your queries in your editor ④.
- ⑥ You can see the outputs of your executed queries here.
- ⑦ If you wish to start over, this Reset button will clear all changes in the tool and datasets. You can reload a dataset from the drop-down menu ① and start over.
- ⑧ This button will allow you to copy your queries in the editor.
- ⑨ This button will allow you to download your queries in the editor into a SQL file.

Exercises

In exercises 1 and 2 you will familiarize yourself with PL/pgSQL to experience the usage and understand the advantages of procedural languages in a DBMS. The lecture on procedural languages provided a thorough overview of PL/pgSQL's constructs, which should be sufficient to complete these

exercises. For additional details or examples, we recommend the official PostgreSQL documentation Chapter ¹.

These exercises use the "Great Outdoors" (GO) Sales dataset from IBM, which contains information on daily sales, sales methods, retailers, and products for a fictitious outdoor equipment retail chain. (Available here : <https://relational.fel.cvut.cz/dataset/GOSales>). Use the PostgreSQL web tool and switch to the "GO Sales" dataset to test and answer the following scenarios.

Given in Figure 2 is the schema of the dataset.

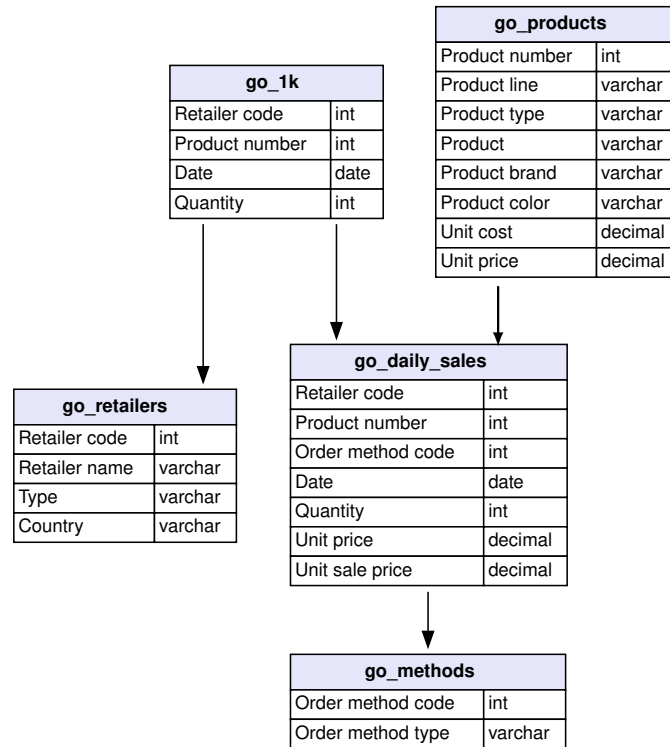


Figure 2: The database schema

Exercise 1 (Functions and Stored procedures) [3 points] The Great Outdoors retail chain wants to create a dashboard showing different aspects of their sales performance for analytics. They want to display the monthly sales performance of a given product for a specific year across all retailers in the dashboard. For this purpose, they need a `get_monthly_sales` function that given a `product_id` and a `year`, returns a table with 12 rows, showing the total sales (income) for each month of that year.

1. Create a function that computes the total sales for all months as follows.
 - For each month, run an aggregation query on `go_daily_sales` to calculate the total sales of that month.
 - If no sales exist for a month, output 0 as the total sales for that month.
2. Rewrite the function using a cursor to improve its efficiency.
3. Discuss the difference in efficiency of the two approaches.

Hint: `EXTRACT` function in PostgreSQL can be used to retrieve specific components (such as year, month, or day) from a date value.

¹<https://www.postgresql.org/docs/>

Exercise 2 (Triggers) [2 points] GO retail chain wants to maintain a `last_sale_date` in the `go_products` table. After adding the column:

```
ALTER TABLE go_products ADD COLUMN last_sale_date DATE;
```

they decide to use triggers to enforce and manage updates. Create triggers for the following scenarios and show how each trigger is attached to its table.

1. The GO team wants to ensure data integrity for the `last_sale_date` column. They need a row-level trigger that fires on every update of `last_sale_date` for a product, with the following behavior:
 - If the new `last_sale_date` is earlier than the existing value, the trigger should raise an exception to prevent the update.
 - If the new date passes the validation, the trigger should log that the update was successful (Use `RAISE NOTICE`) and allow the update to proceed. Note that in the case where there is no previous date, this update should be allowed
2. The GO team wants to automatically maintain the `last_sale_date` in `go_products` whenever new sales are recorded in `go_daily_sales`. Since sales are often inserted in bulk, multiple tuples may be added with a single `INSERT` statement. Create a statement-level trigger that fires once per `INSERT` statement on `go_daily_sales`. The trigger should, update the `last_sale_date` in `go_products` to the current date for all products that have new sales in the inserted rows and log the start and end of the execution of the trigger for auditing purposes (Use `RAISE NOTICE`).
3. Execute single inserts and bulk inserts into `go_daily_sales` and observe the behaviour of the triggers. Explain the behaviour in terms of which triggers fire, the order in which they execute, and how often each trigger fires.